ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

1 Introduction

Current approaches to object recognition make essential use of machine learning methods. To improve their performance, we can collect larger datasets, learn more powerful models, and use better techniques for preventing overfitting. Until recently, datasets of labeled images were relatively small — on the order of tens of thousands of images (e.g., NORB [16], Caltech-101/256 [8, 9], and CIFAR-10/100 [12]). Simple recognition tasks can be solved quite well with datasets of this size, especially if they are augmented with label-preserving transformations. For example, the current-best error rate on the MNIST digit-recognition task (<0.3%) approaches human performance [4]. But objects in realistic settings exhibit considerable variability, so to learn to recognize them it is necessary to use much larger training sets. And indeed, the shortcomings of small image datasets have been widely recognized (e.g., Pinto et al. [21]), but it has only recently become possible to collect labeled datasets with millions of images. The new larger datasets include LabelMe [23], which consists of hundreds of thousands of fully-segmented images, and ImageNet [6], which consists of over 15 million labeled high-resolution images in over 22,000 categories.

To learn about thousands of objects from millions of images, we need a model with a large learning capacity. However, the immense complexity of the object recognition task means that this problem cannot be specified even by a dataset as large as ImageNet, so our model should also have lots of prior knowledge to compensate for all the data we don't have. Convolutional neural networks (CNNs) constitute one such class of models [16, 11, 13, 18, 15, 22, 26]. Their capacity can be controlled by varying their depth and breadth, and they also make strong and mostly correct assumptions about the nature of images (namely, stationarity of statistics and locality of pixel dependencies). Thus, compared to standard feedforward neural networks with similarly-sized layers, CNNs have much fewer connections and parameters and so they are easier to train, while their theoretically-best performance is likely to be only slightly worse.

Classificação ImageNet com Redes Neurais Convolucionais Profundas

Alex Krizhevsky
Universidade de Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
Universidade de Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
Universidade de Toronto
hinton@cs.utoronto.ca

Resumo

Treinamos uma rede neural convolucional grande e profunda para classificar 1,2 milhão de imagens de alta resolução no concurso ImageNet LSVRC-2010 em 1.000 classes diferentes. Nos dados de teste, alcançamos taxas de erro top 1 e top 5 de 37,5% e 17,0%, o que é consideravelmente melhor do que o estado da arte anterior. A rede neural, que te m 60 milhões de parâmetros e 650.000 neurônios, consiste em cinco camadas convolucionais, algumas das quais são seguidas por camadas de pooling máximo, e três camadas totalmente conectadas com um softmax final de 1.000 vias. Para tornar o treinamento mais rápido, usamos neurônios não saturados e uma implementação de GPU muito eficiente da operação de convolução. Para reduzir o overfitting nas camadas totalmente conectadas, empregamos um método de regularização recentemente desenvolvido chamado "dropout" que provou ser muito eficaz. Também inscrevemos uma variante deste modelo na competição ILSVRC-2012 e alcançamos uma taxa de erro de teste entre os 5 primeiros vencedores de 15,3%, em comparação com 26,2% alcançados pelo segundo melhor participante.

1 Introdução

As abordagens atuais para reconhecimento de objetos fazem uso essencial de métodos de aprendizado de máquina. Para melhorar o seu desempenho, podemos coletar conjuntos de dados maiores, aprender modelos mais poderosos e usar melhores técnicas para prevenir o sobreajuste. Até recentemente, os conjuntos de dados de imagens rotuladas eram relativamente pequenos – da ordem de dezenas de milhares de imagens (por exemplo, NORB [16], Caltech-101/256 [8, 9], e CIFAR-10/100 [12]). Tarefas simples de reconhecimento podem ser resolvidas muito bem com conjuntos de dados deste tamanho, especialmente se forem a umentados com transformações que preservam rótulos. Por exemplo, a melhor taxa de erro atual na tarefa de reconhecimento de dígitos MNIST (<0,3%) se aproxima do desempenho humano [4]. Mas os objectos em cenários realistas apresentam uma variabilidade considerável, pelo que para aprender a reconhecê-los é necessário utilizar conjuntos de treino muito maiores. E, de facto, as deficiências de pequenos conjuntos de dados de imagens têm sido amplamente reconhecidas (por exemplo, Pinto et al. [21]), mas só recentemente se tornou possível recolher conjuntos de dados rotulados com milhões de imagens. Os novos conjuntos de dados maiores incluem LabelMe [23], , que consiste em centenas de milhares de imagens totalmente segmentadas, e ImageNet [6], , que consiste em mais de 15 milhões de imagens rotuladas de alta resolução em mais de 22.000 categorias.

Para aprender sobre milhares de objetos a partir de milhões de imagens, precisamos de um modelo com grande capacidade de aprendizagem. No entanto, a imensa complexidade da tarefa de reconhecimento de objetos significa que este problema não pode ser especificado nem mesmo por um conjunto de dados tão grande como o ImageNet, portanto o nosso modelo também deve ter muito conhecimento prévio para compensar todos os dados que não temos. Redes neurais convolucionais (CNNs) constituem uma dessas classes de modelos [16, 11, 13, 18, 15, 22, 26]. A sua capacidade pode ser controlada variando a sua profundidade e amplitude, e também fazem suposições fortes e, na sua maioria, correctas sobre a natureza das imagens (nomeadamente, a estacionariedade das estatísticas e a localização das dependências dos pixels). Assim, em comparação com redes neurais feedforward padrão com camadas de tamanho semelhante, as CNNs têm muito menos conexões e parâmetros e, portanto, são mais fáceis de treinar, enquanto seu desempenho teoricamente melhor provavelmente será apenas um pouco pior.

Despite the attractive qualities of CNNs, and despite the relative efficiency of their local architecture, they have still been prohibitively expensive to apply in large scale to high-resolution images. Luckily, current GPUs, paired with a highly-optimized implementation of 2D convolution, are powerful enough to facilitate the training of interestingly-large CNNs, and recent datasets such as ImageNet contain enough labeled examples to train such models without severe overfitting.

The specific contributions of this paper are as follows: we trained one of the largest convolutional neural networks to date on the subsets of ImageNet used in the ILSVRC-2010 and ILSVRC-2012 competitions [2] and achieved by far the best results ever reported on these datasets. We wrote a highly-optimized GPU implementation of 2D convolution and all the other operations inherent in training convolutional neural networks, which we make available publicly[1]. Our network contains a number of new and unusual features which improve its performance and reduce its training time, which are detailed in Section 3. The size of our network made overfitting a significant problem, even with 1.2 million labeled training examples, so we used several effective techniques for preventing overfitting, which are described in Section 4. Our final network contains five convolutional and three fully-connected layers, and this depth seems to be important: we found that removing any convolutional layer (each of which contains no more than 1% of the model's parameters) resulted in inferior performance.

In the end, the network's size is limited mainly by the amount of memory available on current GPUs and by the amount of training time that we are willing to tolerate. Our network takes between five and six days to train on two GTX 580 3GB GPUs. All of our experiments suggest that our results can be improved simply by waiting for faster GPUs and bigger datasets to become available.

2 The Dataset

ImageNet is a dataset of over 15 million labeled high-resolution images belonging to roughly 22,000 categories. The images were collected from the web and labeled by human labelers using Amazon's Mechanical Turk crowd-sourcing tool. Starting in 2010, as part of the Pascal Visual Object Challenge, an annual competition called the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) has been held. ILSVRC uses a subset of ImageNet with roughly 1000 images in each of 1000 categories. In all, there are roughly 1.2 million training images, 50,000 validation images, and 150,000 testing images.

ILSVRC-2010 is the only version of ILSVRC for which the test set labels are available, so this is the version on which we performed most of our experiments. Since we also entered our model in the ILSVRC-2012 competition, in Section 6 we report our results on this version of the dataset as well, for which test set labels are unavailable. On ImageNet, it is customary to report two error rates: top-1 and top-5, where the top-5 error rate is the fraction of test images for which the correct label is not among the five labels considered most probable by the model.

ImageNet consists of variable-resolution images, while our system requires a constant input dimensionality. Therefore, we down-sampled the images to a fixed resolution of 256×256. Given a rectangular image, we first rescaled the image such that the shorter side was of length 256, and then cropped out the central 256×256 patch from the resulting image. We did not pre-process the images in any other way, except for subtracting the mean activity over the training set from each pixel. So we trained our network on the (centered) raw RGB values of the pixels.

3 The Architecture

The architecture of our network is summarized in Figure 2. It contains eight learned layers — five convolutional and three fully-connected. Below, we describe some of the novel or unusual features of our network's architecture. Sections 3.1-3.4 are sorted according to our estimation of their importance, with the most important first.

[1] http://code.google.com/p/cuda-convnet/

Apesar das qualidades atraentes das CNNs e da relativa eficiência de sua arquitetura local, elas ainda têm si do proibitivamente caras para serem aplicadas em larga escala em imagens de alta resolução. Felizmente, as GPUs atuais, combinadas com uma implementação altamente otimizada de convolução 2D, são poderosas o suficiente para facilitar o treinamento de CNNs interessantemente grandes, e conjuntos de dados recentes, como o ImageNet, contêm exemplos rotulados suficientes para treinar tais modelos sem overfitting severo.

As contribuições específicas deste artigo são as seguintes: treinamos uma das maiores redes neurais convolucionais até o momento nos subconjuntos do ImageNet usados nas competições ILSVRC-2010 e ILSVRC-2012 [2] e alcançamos de longe os melhores resultados já relatados nesses conjuntos de dados. Escrevemos uma implementação de GPU altamente otimizada de convolução 2D e todas as outras operações inerentes ao treinamento de redes neurais convolucionais, que disponibilizamos publicamente[1]. Nossa rede contém uma série de recursos novos e incomuns que melhoram seu desempenho e reduzem seu tempo de treinamento, que são detalhados na Seção 3. O tamanho de nossa rede tornou o overfitting um problema significativo, mesmo com 1,2 milhão de exemplos de treinamento rotulados, então usamos várias técnicas eficazes para prevenir o overfitting, descritas na Seção 4. Nossa rede final contém cinco camadas convolucionais e três totalmente conectadas, e essa profundidade parece ser importante: descobrimos que a remoção de qualquer camada convolucional (cada uma contendo não mais que 1% dos parâmetros do modelo) resultou em desempenho inferior.

No final das contas, o tamanho da rede é limitado principalmente pela quantidade de memória disponível nas GPUs atuais e pela quantidade de tempo de treinamento que estamos dispostos a tolerar. Nossa rede leva de cinco a seis dias para treinar em duas GPUs GTX 580 de 3 GB. Todos os nossos experimentos sugerem que nossos resultados podem ser melhorados simplesmente aguardando a disponibilização de GPUs mais rápidas e conjuntos de dados maiores.

2 O conjunto de dados

ImageNet é um conjunto de dados de mais de 15 milhões de imagens rotuladas de alta resolução pertencentes a aproximadamente 22.000 categorias. As imagens foram coletadas da web e rotuladas por rotuladores humanos usando a ferramenta de crowdsourcing Mechanical Turk da Amazon. A partir de 2010, como parte do Pascal Visual Object Challenge, foi realizada uma competição anual chamada ImageNet Large-Scale Visual Recognition Challenge (ILSVRC). ILSVRC usa um subconjunto do ImageNet com aproximadamente 1.000 imagens em cada uma das 1.000 categorias. Ao todo, existem cerca de 1,2 milhão de imagens de treinamento, 50 mil imagens de validação e 150 mil imagens de teste.

ILSVRC-2010 é a única versão do ILSVRC para a qual os rótulos do conjunto de testes estão disponíveis, portanto esta é a versão na qual realizamos a maioria de nossos experimentos. Como também inscrevemos nosso modelo na competição ILSVRC-2012, na Seção 6 relatamos nossos resultados também nesta versão do conjunto de dados, para a qual os rótulos do conjunto de testes não estão disponíveis. No ImageNet, costuma-se reportar duas taxas de erro: top-1 e top-5, onde a taxa de erro top-5 é a fração de imagens de teste para as quais o rótulo correto não está entre os cinco rótulos considerados mais prováveis pelo modelo.

ImageNet consiste em imagens de resolução variável, enquanto nosso sistema requer uma dimensionalidade de entrada constante. Portanto, reduzimos a amostragem das imagens para uma resolução fixa de 256×256. Dada uma imagem retangular, primeiro redimensionamos a imagem de modo que o lado mais curto tivesse comprimento 256 e, em seguida, recortamos o patch 256×256 central da imagem resultante. Não pré-processamos as imagens de nenhuma outra forma, exceto subtraindo a atividade média do conjunto de treinamento de cada pixel. Portanto, treinamos nossa rede nos valores RGB brutos (centralizados) dos pixels.

3 A Arquitetura

A arquitetura da nossa rede está resumida na Figura 2. Ela contém oito camadas aprendidas – cinco convolucionais e três totalmente conectadas. Abaixo, descrevemos alguns dos recursos novos ou incomuns da arquitetura de nossa rede. As seções 3.1-3.4 são classificadas de acordo com nossa estimativa de importância, com as mais importantes primeiro.

[1] http://code.google.com/p/cuda-convnet/

3.1 ReLU Nonlinearity

The standard way to model a neuron's output f as a function of its input x is with $f(x) = \tanh(x)$ or $f(x) = (1 + e^{-x})^{-1}$. In terms of training time with gradient descent, these saturating nonlinearities are much slower than the non-saturating nonlinearity $f(x) = \max(0, x)$. Following Nair and Hinton [20], we refer to neurons with this nonlinearity as Rectified Linear Units (ReLUs). Deep convolutional neural networks with ReLUs train several times faster than their equivalents with $\tanh$ units. This is demonstrated in Figure 1, which shows the number of iterations required to reach 25% training error on the CIFAR-10 dataset for a particular four-layer convolutional network. This plot shows that we would not have been able to experiment with such large neural networks for this work if we had used traditional saturating neuron models.

We are not the first to consider alternatives to traditional neuron models in CNNs. For example, Jarrett et al. [11] claim that the nonlinearity $f(x) = |\tanh(x)|$ works particularly well with their type of contrast normalization followed by local average pooling on the Caltech-101 dataset. However, on this dataset the primary concern is preventing overfitting, so the effect they are observing is different from the accelerated ability to fit the training set which we report when using ReLUs. Faster learning has a great influence on the performance of large models trained on large datasets.

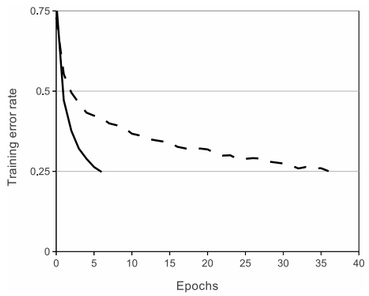


Figure 1: A four-layer convolutional neural network with ReLUs **(solid line)** reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with $\tanh$ neurons **(dashed line)**. The learning rates for each network were chosen independently to make training as fast as possible. No regularization of any kind was employed. The magnitude of the effect demonstrated here varies with network architecture, but networks with ReLUs consistently learn several times faster than equivalents with saturating neurons.

3.2 Training on Multiple GPUs

A single GTX 580 GPU has only 3GB of memory, which limits the maximum size of the networks that can be trained on it. It turns out that 1.2 million training examples are enough to train networks which are too big to fit on one GPU. Therefore we spread the net across two GPUs. Current GPUs are particularly well-suited to cross-GPU parallelization, as they are able to read from and write to one another's memory directly, without going through host machine memory. The parallelization scheme that we employ essentially puts half of the kernels (or neurons) on each GPU, with one additional trick: the GPUs communicate only in certain layers. This means that, for example, the kernels of layer 3 take input from all kernel maps in layer 2. However, kernels in layer 4 take input only from those kernel maps in layer 3 which reside on the same GPU. Choosing the pattern of connectivity is a problem for cross-validation, but this allows us to precisely tune the amount of communication until it is an acceptable fraction of the amount of computation.

The resultant architecture is somewhat similar to that of the "columnar" CNN employed by Cireşan et al. [5], except that our columns are not independent (see Figure 2). This scheme reduces our top-1 and top-5 error rates by 1.7% and 1.2%, respectively, as compared with a net with half as many kernels in each convolutional layer trained on one GPU. The two-GPU net takes slightly less time to train than the one-GPU net[2].

[2]The one-GPU net actually has the same number of kernels as the two-GPU net in the final convolutional layer. This is because most of the net's parameters are in the first fully-connected layer, which takes the last convolutional layer as input. So to make the two nets have approximately the same number of parameters, we did not halve the size of the final convolutional layer (nor the fully-conneced layers which follow). Therefore this comparison is biased in favor of the one-GPU net, since it is bigger than "half the size" of the two-GPU net.

3.1 Não linearidade ReLU

A maneira padrão de modelar a saída de um neurônio f em função de sua entrada x é com $f(x) = \tanh(x)$ ou $f(x) = (1 + e^{-x})^{-1}$. Em termos de tempo de treinamento com descida gradiente, essas não linearidades saturantes são muito mais lentas do que a não linearidade não saturante $f(x) = \max(0, x)$. Seguindo Nair e Hinton [20], nos referimos a os neurônios com essa não linearidade como Unidades Lineares Retificadas (ReLUs). Redes neurais convolucionais profundas com ReLUs treinam várias vezes mais rápido do que suas equivalentes com unidades tanh. Isso é demonstrado na Figura 1, que mostra o número de iterações necessárias para atingir 25% de erro de treinamento no conjunto de dados CIFAR-10 para uma rede convolucional de quatro camadas específica. Este gráfico mostra que não teríamos sido capazes de experimentar redes neurais tão grandes para este trabalho se tivéssemos usado modelos tradicionais de neurônios de saturação.

Não somos os primeiros a considerar alternativas aos modelos tradicionais de neurônios em CNNs. Por exemplo, Jarrett et al. [11] afirmam que a não linearidade $f(x) = |\tanh(x)|$ funciona particularmente bem com seu tipo de normalização de contraste seguido pelo agrupamento de média local no conjunto de dados Caltech-101. No entanto, neste conjunto de dados, a principal preocupação é evitar o sobreajuste, pelo que o efeito que estão a observar é diferente da capacidade acelerada de ajuste ao conjunto de treino que reportamos quando utilizamos ReLUs. O aprendizado mais rápido tem grande influência no desempenho de grandes modelos treinados em grandes conjuntos de dados.

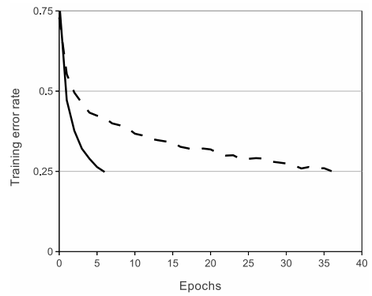


Figura 1: Uma rede neural convolucional de quatro camadas com ReLUs (linha sólida) atinge uma taxa de erro de treinamento de 25% no CIFAR-10 seis vezes mais rápido do que uma rede equivalente co m neurônios tanh (linha tracejada). As taxas de aprendizagem para cada rede foram escolhidas de forma independente para tornar o treinamento o mais rápido possível. Nenhuma regularização de qualquer tipo foi empregada. A magnitude do efeito demonstrado aqui varia com a arquitetura da rede, mas redes com ReLUs aprendem consistentemente várias vezes mais rápido do que equivalentes com neurônios saturados.

3.2 Treinamento em múltiplas GPUs

Uma única GPU GTX 580 possui apenas 3 GB de memória, o que limita o tamanho máximo das redes que podem ser treinadas nela. Acontece que 1,2 milhão de exemplos de treinamento são suficientes para treinar redes que são grandes demais para caber em uma GPU. Portanto, espalhamos a rede por duas GPUs. As GPUs atuais são particularmente adequadas para paralelização entre GPUs, pois são capazes de ler e gravar diretamente na memória umas das outras, sem passar pela memória da máquina host. O esquema de paralelização que empregamos basicamente coloca metade dos núcleos (ou neurônios) em cada GPU, com u m truque adicional: as GPUs se comunicam apenas em determinadas camadas. Isso significa que, por exemplo, os kernels da camada 3 recebem informações de todos os mapas de kernel na camada 2. No entanto, os kernels na camada 4 recebem informações apenas dos mapas de kernel na camada 3 que residem na mesma GPU. A escolha do padrão de conectividade é um problema para validação cruzada, mas isso nos permite ajustar com precisão a quantidade de comunicação até que seja uma fração aceitável da quantidade de computação.

A arquitetura resultante é um pouco semelhante à da CNN "colunar" empregada por Cire.san et al. [5], exceto que nossas colunas não são independentes (veja a Figura 2). Este esquema reduz nossas taxas de erro top 1 e top 5 em 1,7% e 1,2%, respectivamente, em comparação com uma rede com metade do número de kernels e m cada camada convolucional treinada em uma GPU. A rede de duas GPUs leva um pouco menos tempo para treinar do que a rede de uma GPU[2].

[2]Na verdade, a rede de uma GPU tem o mesmo número de kernels que a rede de duas GPUs na camada convolucional final. Isso ocorre porque a maioria dos parâmetros da rede está na primeira camada totalmente conectada, que recebe a última camada convolucional como entrada. Portanto, para fazer com que as duas redes tenham aproximadamente o mesmo número de parâmetros, não reduzimos pela metade o tamanho da camada convolucional final (nem das camadas totalmente conectadas que se seguem). Portanto, esta comparação é tendenciosa a favor da rede de uma GPU, uma vez que é maior que "metade do tamanho" da rede de duas GPUs.

3.3 Local Response Normalization

ReLUs have the desirable property that they do not require input normalization to prevent them from saturating. If at least some training examples produce a positive input to a ReLU, learning will happen in that neuron. However, we still find that the following local normalization scheme aids generalization. Denoting by $a^i_{x,y}$ the activity of a neuron computed by applying kernel i at position (x, y) and then applying the ReLU nonlinearity, the response-normalized activity $b^i_{x,y}$ is given by the expression

$$b^i_{x,y} = a^i_{x,y} / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^\beta$$

where the sum runs over n "adjacent" kernel maps at the same spatial position, and N is the total number of kernels in the layer. The ordering of the kernel maps is of course arbitrary and determined before training begins. This sort of response normalization implements a form of lateral inhibition inspired by the type found in real neurons, creating competition for big activities amongst neuron outputs computed using different kernels. The constants k, n, α, and β are hyper-parameters whose values are determined using a validation set; we used $k = 2$, $n = 5$, $\alpha = 10^{-4}$, and $\beta = 0.75$. We applied this normalization after applying the ReLU nonlinearity in certain layers (see Section 3.5).

This scheme bears some resemblance to the local contrast normalization scheme of Jarrett et al. [11], but ours would be more correctly termed "brightness normalization", since we do not subtract the mean activity. Response normalization reduces our top-1 and top-5 error rates by 1.4% and 1.2%, respectively. We also verified the effectiveness of this scheme on the CIFAR-10 dataset: a four-layer CNN achieved a 13% test error rate without normalization and 11% with normalization[3].

3.4 Overlapping Pooling

Pooling layers in CNNs summarize the outputs of neighboring groups of neurons in the same kernel map. Traditionally, the neighborhoods summarized by adjacent pooling units do not overlap (e.g., [17, 11, 4]). To be more precise, a pooling layer can be thought of as consisting of a grid of pooling units spaced s pixels apart, each summarizing a neighborhood of size $z \times z$ centered at the location of the pooling unit. If we set $s = z$, we obtain traditional local pooling as commonly employed in CNNs. If we set $s < z$, we obtain overlapping pooling. This is what we use throughout our network, with $s = 2$ and $z = 3$. This scheme reduces the top-1 and top-5 error rates by 0.4% and 0.3%, respectively, as compared with the non-overlapping scheme $s = 2, z = 2$, which produces output of equivalent dimensions. We generally observe during training that models with overlapping pooling find it slightly more difficult to overfit.

3.5 Overall Architecture

Now we are ready to describe the overall architecture of our CNN. As depicted in Figure 2, the net contains eight layers with weights; the first five are convolutional and the remaining three are fully-connected. The output of the last fully-connected layer is fed to a 1000-way softmax which produces a distribution over the 1000 class labels. Our network maximizes the multinomial logistic regression objective, which is equivalent to maximizing the average across training cases of the log-probability of the correct label under the prediction distribution.

The kernels of the second, fourth, and fifth convolutional layers are connected only to those kernel maps in the previous layer which reside on the same GPU (see Figure 2). The kernels of the third convolutional layer are connected to all kernel maps in the second layer. The neurons in the fully-connected layers are connected to all neurons in the previous layer. Response-normalization layers follow the first and second convolutional layers. Max-pooling layers, of the kind described in Section 3.4, follow both response-normalization layers as well as the fifth convolutional layer. The ReLU non-linearity is applied to the output of every convolutional and fully-connected layer.

The first convolutional layer filters the $224 \times 224 \times 3$ input image with 96 kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels (this is the distance between the receptive field centers of neighboring

[3] We cannot describe this network in detail due to space constraints, but it is specified precisely by the code and parameter files provided here: http://code.google.com/p/cuda-convnet/.

3.3 Normalização LocalResponse

ReLUs têm a propriedade desejável de não exigirem normalização de entrada para evitar a saturação. Se pelo menos alguns exemplos de treinamento produzirem uma entrada positiva para um ReLU, o aprendizado acontecerá naquele neurônio. No entanto, ainda descobrimos que o seguinte esquema de normalização local auxilia na generalização. Denotando por $a^i_{x,y}$ a atividade de um neurônio computada aplicando o kernel i na posição (x, y) e depois aplicando a não linearidade ReLU, a atividade normalizada de resposta $b^i_{x,y}$ é dada pela expressão

$$b^i_{x,y} = a^i_{x,y} / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^\beta$$

onde a soma percorre n mapas de kernel "adjacentes" na mesma posição espacial e N é o número total de kernels na camada. A ordem dos mapas do kernel é obviamente arbitrária e determinada antes do início do treinamento. Este tipo de normalização de resposta implementa uma forma de inibição lateral inspirada no tipo encontrado em neurônios reais, criando competição por grandes atividades entre saídas de neurônios computadas usando diferentes kernels. As constantes k, n, α e β são hiperparâmetros cujos valores são determinados usando um conjunto de validação; usamos $k = 2$, $n = 5$, $\alpha = 10^{-4}$ e $\beta = 0.75$. Aplicamos esta normalização após aplicar a não linearidade ReLU em certas camadas (ver Seção 3.5).

Este esquema tem alguma semelhança com o esquema de normalização de contraste local de Jarrett et al. [11],mas a nossa seria mais corretamente chamada de "normalização de brilho", já que não subtraímos a atividade média. A normalização de resposta reduz nossas taxas de erro principais 1 e 5 principais em 1,4% e 1,2%, respectivamente. Também verificamos a eficácia deste esquema no conjunto de dados CIFAR-10: uma CNN de quatro camadas alca nçou uma taxa de erro de teste de 13% sem normalização e 11% com normalização[3].

3.4 Pool sobreposto

As camadas de agrupamento em CNNs resumem as saídas de grupos vizinhos de neurônios no mesmo mapa de kernel. Tradicionalmente, as vizinhanças resumidas por unidades de agrupamento adjacentes não se sobrepõem (por exemplo, [17, 11, 4]). Para ser mais preciso, uma camada de pooling pode ser considerada como consistindo de uma grade de unidades de pooling espaçadas em s pixels, cada uma resumindo uma vizinhança de tamanho $z \times z$ centrada no local da unidade de pooling. Se definirmos $s = z$, obteremos o pooling local tradicional comumente empregado em CNNs. Se definirmos $s < z$, obteremos pooling sobreposto. É isso que usamos em toda a nossa rede, com $s = 2$ e $z = 3$. Este esquema reduz as taxas de erro top-1 e top-5 em 0,4% e 0,3%, respectivamente, em comparação com o esquema não sobreposto $s = 2, z = 2$, que produz resultados de dimensões equivalentes. Geralmente observamos durante o treinamento que os modelos com pooling sobreposto têm um pouco mais de dificuldade de superajuste.

3.5 Arquitetura Geral

Agora estamos prontos para descrever a arquitetura geral da nossa CNN. Conforme ilustrado na Figura 2, a rede contém oito camadas com pesos; os primeiros cinco são convolucionais e os três restantes são total mente conectados. A saída da última camada totalmente conectada é alimentada para um softmax de 1000 vias que produz uma distribuição sobre os 1000 rótulos de classe. Nossa rede maximiza o objetivo de regressão logística multinomial, o que equivale a maximizar a média entre os casos de treinamento da probabilidade logarítmica do rótulo correto sob a distribuição de predição.

Os kernels da segunda, quarta e quinta camadas convolucionais são conectados apenas aos mapas de kernel da camada anterior que residem na mesma GPU (veja a Figura 2). Os kernels da terceira camada convolucional estão conectados a todos os mapas de kernel da segunda camada. Os neurônios nas camadas totalmente conectadas estão conectados a todos os neurônios da camada anterior. As camadas de normalização de resposta seguem a primeira e a segunda camadas convolucionais. As camadas de pooling máximo, do tipo descrito na Seção 3.4, seguem ambas as camadas de normalização de resposta, bem como a quinta camada convolucional. A não linearidade ReLU é aplicada à saída de cada camada convolucional e totalmente conectada.

The a primeira camada convolucional filtra a $224 \times 224 \times 3$ imagem de entrada com 96 kernels de tamanho $11 \times 11 \times$ 3 with um passo de 4 pixels (esta é a distância entre os centros do campo receptivo dos vizinhos g

[3] Não podemos descrever esta rede em detalhes devido a restrições de espaço, mas ela é especificada precisamente pelos arquivos de código e parâmetros fornecidos aqui: http://code.google.com/p/cuda-convnet/.

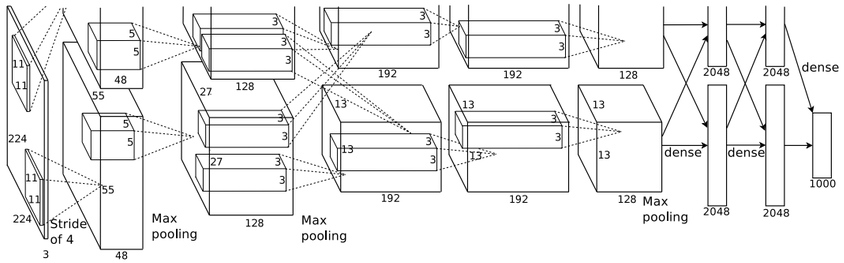


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

neurons in a kernel map). The second convolutional layer takes as input the (response-normalized and pooled) output of the first convolutional layer and filters it with 256 kernels of size $5 \times 5 \times 48$. The third, fourth, and fifth convolutional layers are connected to one another without any intervening pooling or normalization layers. The third convolutional layer has 384 kernels of size $3 \times 3 \times 256$ connected to the (normalized, pooled) outputs of the second convolutional layer. The fourth convolutional layer has 384 kernels of size $3 \times 3 \times 192$, and the fifth convolutional layer has 256 kernels of size $3 \times 3 \times 192$. The fully-connected layers have 4096 neurons each.

4 Reducing Overfitting

Our neural network architecture has 60 million parameters. Although the 1000 classes of ILSVRC make each training example impose 10 bits of constraint on the mapping from image to label, this turns out to be insufficient to learn so many parameters without considerable overfitting. Below, we describe the two primary ways in which we combat overfitting.

4.1 Data Augmentation

The easiest and most common method to reduce overfitting on image data is to artificially enlarge the dataset using label-preserving transformations (e.g., [25, 4, 5]). We employ two distinct forms of data augmentation, both of which allow transformed images to be produced from the original images with very little computation, so the transformed images do not need to be stored on disk. In our implementation, the transformed images are generated in Python code on the CPU while the GPU is training on the previous batch of images. So these data augmentation schemes are, in effect, computationally free.

The first form of data augmentation consists of generating image translations and horizontal reflections. We do this by extracting random 224×224 patches (and their horizontal reflections) from the 256×256 images and training our network on these extracted patches[4]. This increases the size of our training set by a factor of 2048, though the resulting training examples are, of course, highly interdependent. Without this scheme, our network suffers from substantial overfitting, which would have forced us to use much smaller networks. At test time, the network makes a prediction by extracting five 224×224 patches (the four corner patches and the center patch) as well as their horizontal reflections (hence ten patches in all), and averaging the predictions made by the network's softmax layer on the ten patches.

The second form of data augmentation consists of altering the intensities of the RGB channels in training images. Specifically, we perform PCA on the set of RGB pixel values throughout the ImageNet training set. To each training image, we add multiples of the found principal components,

[4]This is the reason why the input images in Figure 2 are $224 \times 224 \times 3$-dimensional.

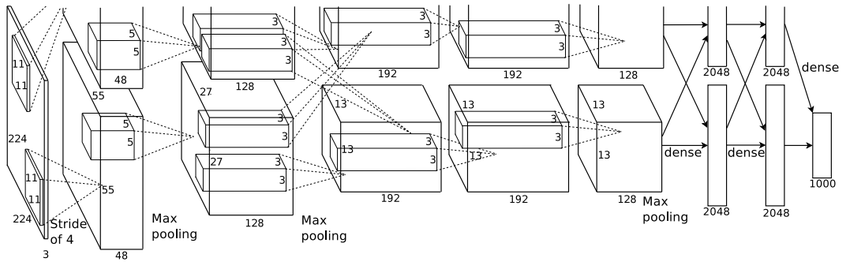


Figura 2: Uma ilustração da arquitetura de nossa CNN, mostrando explicitamente a delimitação de responsabilidades entre as duas GPUs. Uma GPU executa as partes da camada na parte superior da figura, enquanto a outra executa as partes da camada na parte inferior. As GPUs se comunicam apenas em determinadas camadas. A entrada da rede tem 150.528 dimensões e o número de neurônios nas camadas restantes da rede é dado por 253.440–186.624–64.896–64.896–43.264–4.096–4.096–1000.

neurônios em um mapa de kernel). A segunda camada convolucional toma como entrada a saída (normalizada por resposta e agrupada) da primeira camada convolucional e a filtra com 256 kernels de tamanho $5 \times 5 \times 48$. A terceira, quarta e quinta camadas convolucionais são conectadas umas às outras sem nenhuma camada intermediária de agrupamento ou normalização. A terceira camada convolucional possui 384 kernels de tamanho $3 \times 3 \times$256 conectados às saídas (normalizadas, agrupadas) da segunda camada convolucional. A quarta camada convolucional possui 384 kernels de tamanho $3 \times 3 \times 192$, e a quinta c amada convolucional possui 256 kernels de tamanho $3 \times 3 \times 192$. As camadas totalmente conectadas possue m 4.096 neurônios cada.

4 Reduzindo o sobreajuste

Nossa arquitetura de rede neural possui 60 milhões de parâmetros. Embora as 1000 classes do ILSVRC façam com que cada exemplo de treinamento imponha 10 bits de restrição no mapeamento da imagem ao rótulo, isso acaba sendo insuficiente para aprender tantos parâmetros sem um overfitting considerável. Abaixo, descrevemos as duas principais maneiras pelas quais combatemos o overfitting.

4.1 Aumento de dados

O método mais fácil e comum para reduzir o sobreajuste nos dados de imagem é ampliar artificialmente o conjunto de dados usando transformações que preservam rótulos (por exemplo, [25, 4, 5]). Empregamos duas formas distintas de aumento de dados, ambas permitindo que imagens transformadas sejam produzidas a partir das imagens originais com muito pouco cálculo, de forma que as imagens transformadas não precisem ser armazenadas em disco. Em nossa implementação, as imagens transformadas são geradas em código Python na CPU enquanto a GPU treina no lote anterior de imagens. Portanto, esses esquemas de aumento de dados são, na verdade, computacionalmente livres.

A primeira forma de aumento de dados consiste em gerar translações de imagens e reflexões horizontais. Fazemos isso extraindo 224×224 patches aleatórios (e seus reflexos horizontais) das256×256 imagens e treinando nossa rede nesses patches extraídos[4]. Isto aumenta o tamanho do nosso conjunto de formação por um factor de 2048, embora os exemplos de formação resultantes sejam, evidentemente, altamente interdependentes. Sem este esquema, a nossa rede sofre de um overfitting substancial, o que nos teria força do a utilizar redes muito mais pequenas. No momento do teste, a rede faz uma previsão extraindo cinco patches 224×224 (os quatro patches de canto e o patch central), bem como seus reflexos horizontais (portanto, dez patches no total), e calculando a média das previsões feitas pela camada softmax da rede nos dez patches.

A segunda forma de aumento de dados consiste em alterar as intensidades dos canais RGB nas imagens de treinamento. Especificamente, realizamos PCA no conjunto de valores de pixels RGB em todo o conjunto de treinamento ImageNet. Para cada imagem de treinamento, adicionamos múltiplos dos componentes principais encontrados,

[4]Esta é a razão pela qual as imagens de entrada na Figura 2 são dimensionais $224 \times 224 \times 3$.

with magnitudes proportional to the corresponding eigenvalues times a random variable drawn from a Gaussian with mean zero and standard deviation 0.1. Therefore to each RGB image pixel $I_{xy} = [I^R_{xy}, I^G_{xy}, I^B_{xy}]^T$ we add the following quantity:

$$[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$$

where $\mathbf{p}_i$ and λ_i are ith eigenvector and eigenvalue of the 3×3 covariance matrix of RGB pixel values, respectively, and α_i is the aforementioned random variable. Each α_i is drawn only once for all the pixels of a particular training image until that image is used for training again, at which point it is re-drawn. This scheme approximately captures an important property of natural images, namely, that object identity is invariant to changes in the intensity and color of the illumination. This scheme reduces the top-1 error rate by over 1%.

4.2 Dropout

Combining the predictions of many different models is a very successful way to reduce test errors [1, 3], but it appears to be too expensive for big neural networks that already take several days to train. There is, however, a very efficient version of model combination that only costs about a factor of two during training. The recently-introduced technique, called "dropout" [10], consists of setting to zero the output of each hidden neuron with probability 0.5. The neurons which are "dropped out" in this way do not contribute to the forward pass and do not participate in back-propagation. So every time an input is presented, the neural network samples a different architecture, but all these architectures share weights. This technique reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons. It is, therefore, forced to learn more robust features that are useful in conjunction with many different random subsets of the other neurons. At test time, we use all the neurons but multiply their outputs by 0.5, which is a reasonable approximation to taking the geometric mean of the predictive distributions produced by the exponentially-many dropout networks.

We use dropout in the first two fully-connected layers of Figure 2. Without dropout, our network exhibits substantial overfitting. Dropout roughly doubles the number of iterations required to converge.

5 Details of learning

We trained our models using stochastic gradient descent with a batch size of 128 examples, momentum of 0.9, and weight decay of 0.0005. We found that this small amount of weight decay was important for the model to learn. In other words, weight decay here is not merely a regularizer: it reduces the model's training error. The update rule for weight w was

$$\begin{aligned} v_{i+1} &:= 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w}\Big|_{w_i} \right\rangle_{D_i} \\ w_{i+1} &:= w_i + v_{i+1} \end{aligned}$$

where i is the iteration index, v is the momentum variable, ϵ is the learning rate, and $\left\langle \frac{\partial L}{\partial w}\big|_{w_i} \right\rangle_{D_i}$ is the average over the ith batch D_i of the derivative of the objective with respect to w, evaluated at w_i.

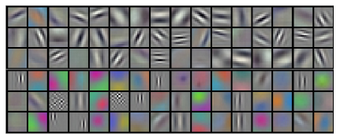


Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

We initialized the weights in each layer from a zero-mean Gaussian distribution with standard deviation 0.01. We initialized the neuron biases in the second, fourth, and fifth convolutional layers, as well as in the fully-connected hidden layers, with the constant 1. This initialization accelerates the early stages of learning by providing the ReLUs with positive inputs. We initialized the neuron biases in the remaining layers with the constant 0.

We used an equal learning rate for all layers, which we adjusted manually throughout training. The heuristic which we followed was to divide the learning rate by 10 when the validation error rate stopped improving with the current learning rate. The learning rate was initialized at 0.01 and

com magnitudes proporcionais aos autovalores correspondentes vezes uma variável aleatória extraída de uma Gaussiana com média zero e desvio padrão 0,1. Portanto, para cada pixel da imagem RGB $I_{xy} = [I^R_{xy}, I^G_{xy}, I^B_{xy}]^T$ adicionamos a seguinte quantidade:

$$[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$$

onde $\mathbf{p}_i$ e λ_i são io autovetor e o autovalor da 3×3 matriz de covariância dos valores de pixel RGB, respectivamente, e α_i é a variável aleatória mencionada acima. Cada α_i é desenhado apenas uma vez para todos os pixels de uma imagem de treinamento específica até que essa imagem seja usada para treinamento novamente, momento em que é redesenhada. Este esquema captura aproximadamente uma propriedade importante das imagens naturais, a saber, que a identidade do objeto é invariante às mudanças na intensidade e na cor da iluminação. Este esquema reduz a taxa de erro top-1 em mais de 1%.

4.2 Desistência

Combinar as previsões de muitos modelos diferentes é uma maneira muito eficaz de reduzir erros de teste, [1, 3], mas parece ser muito caro para grandes redes neurais que já levam vários dias para serem treinadas. Existe, no entanto, uma versão muito eficiente de combinação de modelos que custa apenas cerca de um fator de dois durante o treinamento. A técnica recentemente introduzida, chamada "dropout" [10], consiste em zerar a saída de cada neurônio oculto com probabilidade 0,5. Os neurônios que são "abandonados" dessa maneira não contribuem para a passagem direta e não participam da retropropagação. Portanto, sempre que uma entrada é apresentada, a rede neural mostra uma arquitetura diferente, mas todas essas arquiteturas compartilham pesos. Esta técnica reduz co-adaptações complexas de neurônios, uma vez que um neurônio não pode contar com a presença de outros neurônios específicos. É, portanto, forçado a aprender recursos mais robustos que são úteis em conjunto com muitos subconjuntos aleatórios diferentes de outros neurônios. No momento do teste, usamos todos os neurônios, mas multiplicamos suas saídas por 0,5, o que é uma aproximação razoável para obter a média geométrica das distribuições preditivas produzidas pelas redes de abandonos exponencialmente.

Usamos dropout nas duas primeiras camadas totalmente conectadas da Figura 2. Sem dropout, nossa rede apresenta overfitting substancial. O abandono praticamente dobra o número de iterações necessárias para convergir.

5 Detalhes da aprendizagem

Treinamos nossos modelos usando descida gradiente estocástica com tamanho de lote de 128 exemplos, momento de 0,9 e queda de peso de 0,0005. Descobrimos que essa pequena quantidade de redução de peso era importante para o aprendizado do modelo. Em outras palavras, a redução de peso aqui não é apenas um regularizador: ela reduz o erro de treinamento do modelo. A regra de atualização para peso w era

$$\begin{aligned} v_{i+1} &:= 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w}\Big|_{w_i} \right\rangle_{D_i} \\ w_{i+1} &:= w_i + v_{i+1} \end{aligned}$$

onde i é o índice de iteração, v é a variável de momento, ϵ é a taxa de aprendizado e $\left\langle \frac{\partial L}{\partial w}\big|_{w_i} \right\rangle_{D_i}$ é a média do i ésimo lote D_i da derivada do objetivo em relação a w, avaliada em w_i.

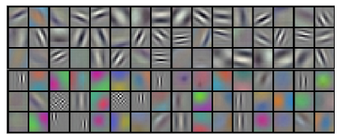


Figura 3: 96 kernels convolucionais de ta manho $11 \times 11 \times 3$ apreendidos pela primeira camada convolucional nas $224 \times 224 \times 3$ imagens de entrada. Os 48 kernels principais foram aprendidos na GPU 1, enquanto os 48 kernels inferiores foram aprendidos na GPU 2. Consulte a Seção 6.1 para obter detalhes.

Inicializamos os pesos em cada camada a partir de uma distribuição gaussiana de média zero com desvio padrão de 0,01. Inicializamos os vieses dos neurônios na segunda, quarta e quinta camadas convolucionais, bem como nas camadas ocultas totalmente conectadas, com a constante 1. Essa inicialização acelera os estágios iniciais de aprendizagem, fornecendo entradas positivas aos ReLUs. Inicializamos os vieses dos neurônios nas camadas restantes com a constante 0.

Usamos uma taxa de aprendizado igual para todas as camadas, que ajustamos manualmente durante o treinamento. A heurística que seguimos foi dividir a taxa de aprendizagem por 10 quando a taxa de erro de validação parou de melhorar com a taxa de aprendizagem atual. A taxa de aprendizagem foi inicializada em 0,01 e

reduced three times prior to termination. We trained the network for roughly 90 cycles through the training set of 1.2 million images, which took five to six days on two NVIDIA GTX 580 3GB GPUs.

6 Results

Our results on ILSVRC-2010 are summarized in Table 1. Our network achieves top-1 and top-5 test set error rates of **37.5%** and **17.0%**[5]. The best performance achieved during the ILSVRC-2010 competition was 47.1% and 28.2% with an approach that averages the predictions produced from six sparse-coding models trained on different features [2], and since then the best published results are 45.7% and 25.7% with an approach that averages the predictions of two classifiers trained on Fisher Vectors (FVs) computed from two types of densely-sampled features [24].

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

Table 1: Comparison of results on ILSVRC-2010 test set. In *italics* are best results achieved by others.

We also entered our model in the ILSVRC-2012 competition and report our results in Table 2. Since the ILSVRC-2012 test set labels are not publicly available, we cannot report test error rates for all the models that we tried. In the remainder of this paragraph, we use validation and test error rates interchangeably because in our experience they do not differ by more than 0.1% (see Table 2). The CNN described in this paper achieves a top-5 error rate of 18.2%. Averaging the predictions of five similar CNNs gives an error rate of 16.4%. Training one CNN, with an extra sixth convolutional layer over the last pooling layer, to classify the entire ImageNet Fall 2011 release (15M images, 22K categories), and then "fine-tuning" it on ILSVRC-2012 gives an error rate of 16.6%. Averaging the predictions of two CNNs that were pre-trained on the entire Fall 2011 release with the aforementioned five CNNs gives an error rate of **15.3%**. The second-best contest entry achieved an error rate of 26.2% with an approach that averages the predictions of several classifiers trained on FVs computed from different types of densely-sampled features [7].

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SIFT + FVs [7]	—	—	*26.2%*
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	**16.4%**
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	**15.3%**

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics* are best results achieved by others. Models with an asterisk* were "pre-trained" to classify the entire ImageNet 2011 Fall release. See Section 6 for details.

Finally, we also report our error rates on the Fall 2009 version of ImageNet with 10,184 categories and 8.9 million images. On this dataset we follow the convention in the literature of using half of the images for training and half for testing. Since there is no established test set, our split necessarily differs from the splits used by previous authors, but this does not affect the results appreciably. Our top-1 and top-5 error rates on this dataset are **67.4%** and **40.9%**, attained by the net described above but with an additional, sixth convolutional layer over the last pooling layer. The best published results on this dataset are 78.1% and 60.9% [19].

6.1 Qualitative Evaluations

Figure 3 shows the convolutional kernels learned by the network's two data-connected layers. The network has learned a variety of frequency- and orientation-selective kernels, as well as various colored blobs. Notice the specialization exhibited by the two GPUs, a result of the restricted connectivity described in Section 3.5. The kernels on GPU 1 are largely color-agnostic, while the kernels on on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization (modulo a renumbering of the GPUs).

[5]The error rates without averaging predictions over ten patches as described in Section 4.1 are 39.0% and 18.3%.

reduzido três vezes antes da rescisão. Treinamos a rede por aproximadamente 90 ciclos por meio do conjunto de treinamento de 1,2 milhão de imagens, o que levou de cinco a seis dias em duas GPUs NVIDIA GTX 580 de 3 GB.

6 resultados

Nossos resultados no ILSVRC-2010 estão resumidos na Tabela 1. Nossa rede atinge taxas de erro dos conjuntos de testes top 1 e top 5 de 37,5% e 17,0%[5]. O melhor desempenho alcançado durante a competição ILSVRC-2010 foi de 47,1% e 28,2% com uma abordagem que calcula a média das previsões produzidas a partir de seis modelos de codificação esparsa treinados em recursos diferentes [2], e desde então os melhores resultados publicados são 45,7% e 25,7% com uma abordagem que calcula a média das previsões de dois classificadores treinados em vetores de Fisher (FVs) calculados a partir de dois tipos de recursos densamente amostrados [24].

Modelo	Primeiro-1	5 primeiros
Codificação esparsa [2]	*47,1%*	*28,2%*
SIFT + FVs [24]	*45,7%*	*25,7%*
CNN	**37,5%**	**17,0%**

Tabela 1: Comparação dos resultados do conjunto de testes ILSVRC-2010. Em itálico estão os melhores resultados alcançados por outros.

Também inscrevemos nosso modelo na competição ILSVRC-2012 e reportamos nossos resultados na Tabela 2. Como os rótulos dos conjuntos de testes ILSVRC-2012 não estão disponíveis publicamente, não podemos relatar taxas de erro de teste para todos os modelos que testamos. No restante deste parágrafo, usamos as taxas de erro de validação e de teste de forma intercambiável porque, em nossa experiência, elas não diferem em mais de 0,1% (ver Tabela 2). A CNN descrita neste artigo atinge uma taxa de erro entre os cinco primeiros de 18,2%. Calculando a média das previsões de cinco CNNs semelhantes dá uma taxa de erro de 16,4%. Treinar uma CNN, com uma sexta camada convolucional extra sobre a última camada de pooling, para classificar todo o lançamento do ImageNet Fall 2011 (15 milhões de imagens, 22 mil categorias) e depois "ajustá-lo" no ILSVRC-2012 resulta em uma taxa de erro de 16,6%. A média das previsões de duas CNNs que foram pré-treinadas em todo o lançamento do outono de 2011 com as cinco CNNs acima mencionadas dá uma taxa de erro de 15,3%. A segunda melhor entrada no concurso alcançou uma taxa de erro de 26,2% com uma abordagem que calcula a média das previsões de vários classificadores treinados em FVs calculados a partir de diferentes tipos de recursos densamente amostrados [7].

Modelo	Top 1 (valor)	5 primeiros (valor)	5 primeiros (teste)
SIFT + FVs [7]	—	—	*26,2%*
1CNN	40,7%	18,2%	—
5 CNNs	38,1%	16,4%	**16,4%**
1CNN*	39,0%	16,6%	—
7 CNN*	36,7%	15,4%	**15,3%**

Tabela 2: Comparação das taxas de erro na validação do ILSVRC-2012 e nos conjuntos de teste. Em itálico estão os melhores resultados alcançados por outros. Os modelos com um asterisco* foram "pré-treinados" para classificar todo o lançamento do ImageNet 2011 Fall. Consulte a Seção 6 para obter detalhes.

Por fim, também relatamos nossas taxas de erro na versão do outono de 2009 do ImageNet, com 10.184 categorias e 8,9 milhões de imagens. Neste conjunto de dados seguimos a convenção da literatura de usar metade das imagens para treina mento e metade para teste. Como não existe um conjunto de testes estabelecido, nossa divisão necessariamente difere das divisões usadas por autores anteriores, mas i sso não afeta os resultados de forma apreciável. Nossas taxas de erro 1 e 5 principais neste conjunto de dados são de 67,4% e 40,9%, alcançado pela rede descrita acima, mas com uma sexta camada convolucional adicional sobre a última camada de pooling. Os melhores resultados publicados neste conjunto de dados são 78,1% e 60,9% [19].

6.1 Avaliações Qualitativas

A Figura 3 mostra os kernels convolucionais aprendidos pelas duas camadas conectadas a dados da rede. A rede aprendeu uma variedade de núcleos seletivos de frequência e orientação, bem como vários blobs coloridos. Observe a especialização exibida pelas duas GPUs, resultado da conectividade restrita descrita na Seção 3.5. Os kernels na GPU 1 são em grande parte independentes de cores, enquanto os kernels na GPU 2 são em grande parte específicos de cores. Esse tipo de especialização ocorre durante cada execução e é independente de qualquer inicialização de peso aleatório específica (módulo uma renumeração das GPUs).

[5]As taxas de erro sem a média das previsões em dez patches, conforme descrito na Seção 4.1, são de 39,0% e 18,3%.

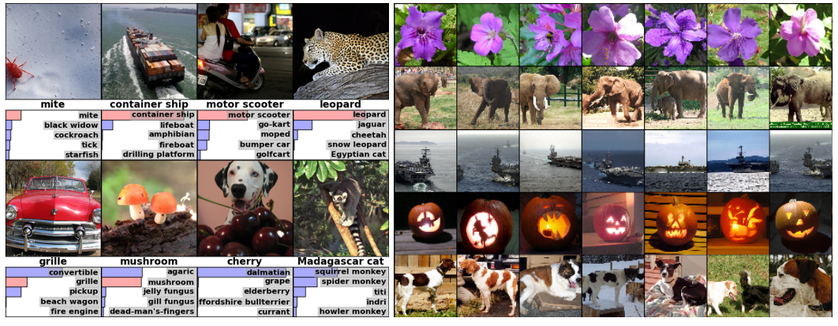


Figure 4: **(Left)** Eight ILSVRC-2010 test images and the five labels considered most probable by our model. The correct label is written under each image, and the probability assigned to the correct label is also shown with a red bar (if it happens to be in the top 5). **(Right)** Five ILSVRC-2010 test images in the first column. The remaining columns show the six training images that produce feature vectors in the last hidden layer with the smallest Euclidean distance from the feature vector for the test image.

In the left panel of Figure 4 we qualitatively assess what the network has learned by computing its top-5 predictions on eight test images. Notice that even off-center objects, such as the mite in the top-left, can be recognized by the net. Most of the top-5 labels appear reasonable. For example, only other types of cat are considered plausible labels for the leopard. In some cases (grille, cherry) there is genuine ambiguity about the intended focus of the photograph.

Another way to probe the network's visual knowledge is to consider the feature activations induced by an image at the last, 4096-dimensional hidden layer. If two images produce feature activation vectors with a small Euclidean separation, we can say that the higher levels of the neural network consider them to be similar. Figure 4 shows five images from the test set and the six images from the training set that are most similar to each of them according to this measure. Notice that at the pixel level, the retrieved training images are generally not close in L2 to the query images in the first column. For example, the retrieved dogs and elephants appear in a variety of poses. We present the results for many more test images in the supplementary material.

Computing similarity by using Euclidean distance between two 4096-dimensional, real-valued vectors is inefficient, but it could be made efficient by training an auto-encoder to compress these vectors to short binary codes. This should produce a much better image retrieval method than applying auto-encoders to the raw pixels [14], which does not make use of image labels and hence has a tendency to retrieve images with similar patterns of edges, whether or not they are semantically similar.

7 Discussion

Our results show that a large, deep convolutional neural network is capable of achieving record-breaking results on a highly challenging dataset using purely supervised learning. It is notable that our network's performance degrades if a single convolutional layer is removed. For example, removing any of the middle layers results in a loss of about 2% for the top-1 performance of the network. So the depth really is important for achieving our results.

To simplify our experiments, we did not use any unsupervised pre-training even though we expect that it will help, especially if we obtain enough computational power to significantly increase the size of the network without obtaining a corresponding increase in the amount of labeled data. Thus far, our results have improved as we have made our network larger and trained it longer but we still have many orders of magnitude to go in order to match the infero-temporal pathway of the human visual system. Ultimately we would like to use very large and deep convolutional nets on video sequences where the temporal structure provides very helpful information that is missing or far less obvious in static images.

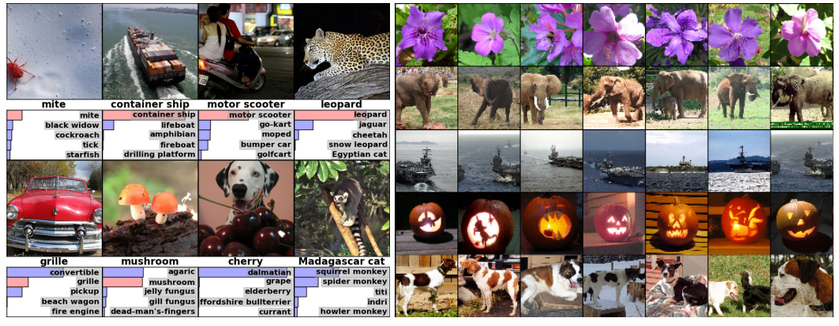


Figura 4: (Esquerda) Oito imagens de teste ILSVRC-2010 e os cinco rótulos considerados mais prováveis pelo nosso modelo. O rótulo correto é escrito abaixo de cada imagem, e a probabilidade atribuída ao rótulo correto também é mostrada com uma barra vermelha (se estiver entre os 5 primeiros). (Direita) Cinco imagens de teste ILSVRC-2010 na primeira coluna. As colunas restantes mostram as seis imagens de treinamento que produzem vetores de características na última camada oculta com a menor distância euclidiana do vetor de características para a imagem de teste.

No painel esquerdo da Figura 4 avaliamos qualitativamente o que a rede aprendeu calculando suas cinco principais previsões em oito imagens de teste. Observe que mesmo objetos descentralizados, como o ácaro no canto superior esquerdo, podem ser reconhecidos pela rede. A maioria dos cinco principais rótulos parece razoável. Por exemplo, apenas outros tipos de gatos são considerados rótulos plausíveis para o leopardo. Em alguns casos (grade, cereja) existe uma ambigüidade genuína sobre o foco pretendido da fotografia.

Outra forma de testar o conhecimento visual da rede é considerar as ativações de recursos induzidas por u ma imagem na última camada oculta de 4.096 dimensões. Se duas imagens produzem vetores de ativação de características com uma pequena separação euclidiana, podemos dizer que os níveis superiores da rede neural os consideram semelhantes. A Figura 4 mostra cinco imagens do conjunto de teste e as seis imagens do conjunto de treinamento que são mais semelhantes a cada uma delas de acordo com esta medida. Observe que no nível do pixel, as imagens de treinamento recuperadas geralmente não estão próximas em L2 das imagens de consulta na primeira coluna. Por exemplo, os cães e elefantes recuperados aparecem em diversas poses. Apresentamos os resultados de muitas outras imagens de teste no material suplementar.

Calcular a similaridade usando a distância euclidiana entre dois vetores de valor real com dimensão 4096 é ineficiente, mas poderia ser tornado eficiente treinando um codificador automático para comprimir esses vetores e m códigos binários curtos. Isso deve produzir um método de recuperação de imagem muito melhor do que aplicar codificadores automáticos aos pixels brutos [14], que não faz uso de rótulos de imagem e, portanto, tem uma tendência a recuperar imagens com padrões de bordas semelhantes, sejam elas semanticamente semelhantes ou não.

7 Discussão

Nossos resultados mostram que uma rede neural convolucional grande e profunda é capaz de alcançar resultados recordes em um conjunto de dados altamente desafiador usando aprendizado puramente supervisionado. É notável que o desempenho da nossa rede diminui se uma única camada convolucional for removida. Por exemplo, a re moção de qualquer uma das camadas intermediárias resulta em uma perda de cerca de 2% no desempenho principal da rede. Então a profundidade é realmente importante para alcançar nossos resultados.

Para simplificar nossos experimentos, não utilizamos nenhum pré-treinamento não supervisionado, embora esperemos que isso ajude, especialmente se obtivermos poder computacional suficiente para aumentar significativamente o tamanho da rede sem obter um aumento correspondente na quantidade de dados rotulados. Até agora, nossos resultados melhoraram à medida que aumentamos nossa rede e a treinamos por mais tempo, mas ainda temos muitas ordens de magnitude pela frente para corresponder ao caminho infero-temporal do sistema visual humano. Em última análise, gostaríamos de usar redes convolucionais muito grandes e profundas em sequências de vídeo onde a estrutura temporal fornece informações muito úteis que estão faltando ou são imagens estáticas muito menos óbvias.

References

[1] R.M. Bell and Y. Koren. Lessons from the netflix prize challenge. *ACM SIGKDD Explorations Newsletter*, 9(2):75–79, 2007.

[2] A. Berg, J. Deng, and L. Fei-Fei. Large scale visual recognition challenge 2010. www.image-net.org/challenges. 2010.

[3] L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[4] D. Cireşan, U. Meier, and J. Schmidhuber. Multi-column deep neural networks for image classification. *Arxiv preprint arXiv:1202.2745*, 2012.

[5] D.C. Cireşan, U. Meier, J. Masci, L.M. Gambardella, and J. Schmidhuber. High-performance neural networks for visual object classification. *Arxiv preprint arXiv:1102.0183*, 2011.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.

[7] J. Deng, A. Berg, S. Satheesh, H. Su, A. Khosla, and L. Fei-Fei. *ILSVRC-2012*, 2012. URL `http://www.image-net.org/challenges/LSVRC/2012/`.

[8] L. Fei-Fei, R. Fergus, and P. Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. *Computer Vision and Image Understanding*, 106(1):59–70, 2007.

[9] G. Griffin, A. Holub, and P. Perona. Caltech-256 object category dataset. Technical Report 7694, California Institute of Technology, 2007. URL `http://authors.library.caltech.edu/7694`.

[10] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R.R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

[11] K. Jarrett, K. Kavukcuoglu, M. A. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? In *International Conference on Computer Vision*, pages 2146–2153. IEEE, 2009.

[12] A. Krizhevsky. Learning multiple layers of features from tiny images. Master's thesis, Department of Computer Science, University of Toronto, 2009.

[13] A. Krizhevsky. Convolutional deep belief networks on cifar-10. *Unpublished manuscript*, 2010.

[14] A. Krizhevsky and G.E. Hinton. Using very deep autoencoders for content-based image retrieval. In *ESANN*, 2011.

[15] Y. Le Cun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, et al. Handwritten digit recognition with a back-propagation network. In *Advances in neural information processing systems*, 1990.

[16] Y. LeCun, F.J. Huang, and L. Bottou. Learning methods for generic object recognition with invariance to pose and lighting. In *Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference on*, volume 2, pages II–97. IEEE, 2004.

[17] Y. LeCun, K. Kavukcuoglu, and C. Farabet. Convolutional networks and applications in vision. In *Circuits and Systems (ISCAS), Proceedings of 2010 IEEE International Symposium on*, pages 253–256. IEEE, 2010.

[18] H. Lee, R. Grosse, R. Ranganath, and A.Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 609–616. ACM, 2009.

[19] T. Mensink, J. Verbeek, F. Perronnin, and G. Csurka. Metric Learning for Large Scale Image Classification: Generalizing to New Classes at Near-Zero Cost. In *ECCV - European Conference on Computer Vision*, Florence, Italy, October 2012.

[20] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proc. 27th International Conference on Machine Learning*, 2010.

[21] N. Pinto, D.D. Cox, and J.J. DiCarlo. Why is real-world visual object recognition hard? *PLoS computational biology*, 4(1):e27, 2008.

[22] N. Pinto, D. Doukhan, J.J. DiCarlo, and D.D. Cox. A high-throughput screening approach to discovering good forms of biologically inspired visual representation. *PLoS computational biology*, 5(11):e1000579, 2009.

[23] B.C. Russell, A. Torralba, K.P. Murphy, and W.T. Freeman. Labelme: a database and web-based tool for image annotation. *International journal of computer vision*, 77(1):157–173, 2008.

[24] J. Sánchez and F. Perronnin. High-dimensional signature compression for large-scale image classification. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1665–1672. IEEE, 2011.

[25] P.Y. Simard, D. Steinkraus, and J.C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, volume 2, pages 958–962, 2003.

[26] S.C. Turaga, J.F. Murray, V. Jain, F. Roth, M. Helmstaedter, K. Briggman, W. Denk, and H.S. Seung. Convolutional networks can learn to generate affinity graphs for image segmentation. *Neural Computation*, 22(2):511–538, 2010.

Referências

[1] RM Bell e Y. Koren. Lições do desafio do prêmio Netflix. Boletim Informativo de Explorações ACM SIGKDD, 9(2):75–79, 2007. [2] A. Berg, J. Deng e L. Fei-Fei. Desafio de reconhecimento visual em grande escala 2010. www.image-net.org/challenges. 2010. [3] EU. Breiman. Florestas aleatórias. Aprendizado de máquina, 45(1):5–32, 2001. [4] D. Cire.san, U. Meier e J. Schmidhuber. Redes neurais profundas multicolunas para classificação de imagens. Pré-impressão do Arxiv arXiv:1202.2745, 2012. [5] D.C. Cire.san, U. Meier, J. Masci, LM Gambardella e J. Schmidhuber. Redes neurais de alto desempenho para classificação visual de objetos. Pré-impressão do Arxiv arXiv:1102.0183, 2011. [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li e L. Fei-Fei. ImageNet: um banco de dados de imagens hierárquicas em grande escala. Em CVPR09, 2009. [7] J. Deng, A. Berg, S. Satheesh, H. Su, A. Khosla e L. Fei-Fei. ILSVRC-2012, 2012. URL http://www.image-net.org/challenges/LSVRC/2012/. [8] EU. Fei-Fei, R. Fergus e P. Perona. Aprendendo modelos visuais generativos a partir de alguns exemplos de treinamento: Uma abordagem bayesiana incremental testada em 101 categorias de objetos. Visão Computacional e Compreensão de Imagens, 106(1):59–70, 2007. [9] G. Griffin, A. Holub e P. Perona. Conjunto de dados de categoria de objeto Caltech-256. Relatório Técnico 7694, Instituto de Tecnologia da Califórnia, 2007. URL http://authors.library.caltech.edu/7694. [10] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever e RR Salakhutdinov. Melhorar as redes neurais evitando a coadaptação de detectores de recursos. Pré-impressão do arXiv arXiv:1207.0580, 2012. [11] K. Jarrett, K. Kavukcuoglu, MA Ranzato e Y. LeCun. Qual é a melhor arquitetura multiestágio para reconhecimento de objetos? Na Conferência Internacional sobre Visão Computacional, páginas 2146–2153. IEEE, 2009. [12] A. Krizhevsky. Aprendendo múltiplas camadas de recursos a partir de imagens minúsculas. Tese de mestrado, Departamento de Ciência da Computação, Universidade de Toronto, 2009. [13] A. Krizhevsky. Redes convolucionais de crenças profundas em cifar-10. Manuscrito não publicado, 2010. [14] A. Krizhevsky e G.E. Hinton. Usando autoencoders muito profundos para recuperação de imagens baseada em conteúdo. Na ESANN, 2011. [15] Y. Le Cun, B. Boser, J.S. Denker, D. Henderson, RE. Howard, W. Hubbard, L.D. Jackel, et al. Reconhecimento de dígitos manuscritos com rede de retropropagação. Em Avanços em sistemas de processamento de informações neurais, 1990. [16] Y. LeCun, FJ Huang e L. Bottou. Métodos de aprendizagem para reconhecimento genérico de objetos com invariância de pose e iluminação. Em Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference o n, volume 2, páginas II–97. IEEE, 2004. [17] Y. LeCun, K. Kavukcuoglu e C. Farabet. Redes convolucionais e aplicações em visão. Em Circuitos e Sistemas (ISCAS), Proceedings of 2010 IEEE International Symposium on, páginas 253–256. IEEE, 2010. [18] H. Lee, R. Grosse, R. Ranganath e A.Y. Ng. Redes convolucionais de crenças profundas para aprendizagem escalonável e não supervisionada de representações hierárquicas. Nos Anais da 26ª Conferência Internacional Anual so bre Aprendizado de Máquina, páginas 609–616. ACM, 2009. [19] T. Mensink, J. Verbeek, F. Perronnin e G. Csurka. Aprendizagem Métrica para Classificação de Imagens em Grande Escala: Generalizando para Novas Classes com Custo Quase Zero. In ECCV - European Conference on Computer Vision, Florença, Itália, outubro de 2012. [20] V. Nair e GE Hinton. Unidades lineares retificadas melhoram máquinas boltzmann restritas. Em Proc. 27ª Conferência Internacional sobre Aprendizado de Máquina, 2010. [21] N. Pinto, D. D. Cox e J.J. DiCarlo. Por que o reconhecimento visual de objetos do mundo real é difícil? Biologia computacional PLoS, 4(1):e27, 2008. [22] N. Pinto, D. Doukhan, J.J. DiCarlo e D.D. Cox. Uma abordagem de triagem de alto rendimento para descobrir boas formas de representação visual de inspiração biológica. Biologia computacional PLoS, 5 1000579, 2009 11( ):e .[23] B.C. Russell, A. Torralba, K.P. Murphy e WT Freeman. Labelme: um banco de dados e uma ferramenta baseada na web para anotação de imagens. Jornal internacional de visão computacional, 77(1):157–173, 2008. [24] J. Sánchez e F. Perronnin. Compressão de assinatura de alta dimensão para classificação de imagens em grande escala. Em Computer Vision and Pattern Recognition (CVPR), Conferência IEEE de 2011 em, páginas 1665–1672. IEEE, 2011.[25] P.Y. Simard, D. Steinkraus e JC Platt. Melhores práticas para redes neurais convolucionais aplicadas à análise visual de documentos. Em Anais da Sétima Conferência Internacional sobre Análise e Reconhecimento de Documentos, volume 2, páginas 958–962, 2003. [26] S.C. Turaga, JF Murray, V. Jain, F. Roth, M. Helmstaedter, K. Briggman, W. Denk e HS. Seung. Redes convolucionais podem aprender a gerar gráficos de afinidade para segmentação de imagens. Computação Neural, 22(2):511–538, 2010.